

Operaciones con Matrices en Python utilizando NumPy y matrix

Documento de Ejemplos Prácticos

15 de septiembre de 2026

Resumen

Este documento presenta una colección de ejemplos prácticos sobre el uso de la librería NumPy en Python para realizar operaciones con matrices. Se abordan desde la creación básica de matrices mediante `matrix` y `array`, hasta operaciones como suma, resta, multiplicación (elemento a elemento y matricial), transposición, cálculo de determinantes, inversas y resolución de sistemas lineales.

Índice

1. Introducción	2
2. Creación de Matrices	2
2.1. Usando <code>np.matrix</code>	2
2.2. Usando <code>np.array</code>	2
2.3. Matrices especiales	3
3. Operaciones Básicas entre Matrices	3
3.1. Suma y resta	3
3.2. Multiplicación elemento a elemento	3
3.3. Multiplicación matricial	4
4. Operaciones Avanzadas	4
4.1. Transpuesta	4
4.2. Determinante	4
4.3. Inversa	5
4.4. Rango de una matriz	5
4.5. Traza	5
5. Resolución de Sistemas de Ecuaciones Lineales	5
6. Ejemplo Integrador	6
7. Resumen de Funciones de NumPy	7
8. Conclusión	7

1. Introducción

NumPy (*Numerical Python*) es una de las librerías más importantes del ecosistema científico de Python. Proporciona estructuras de datos eficientes para trabajar con arreglos y matrices, así como una gran cantidad de funciones matemáticas.

Para usar NumPy es necesario importarla:

```
1 import numpy as np
```

Listing 1: Importación de NumPy

En este documento se mostrarán ejemplos utilizando tanto `np.matrix` (subclase especializada para álgebra lineal 2D) como `np.array` (la forma recomendada actualmente).

Nota importante

Aunque `np.matrix` todavía está disponible, **NumPy recomienda usar `np.array` con el operador `@`** para la multiplicación matricial, ya que `np.matrix` está en proceso de desuso.

2. Creación de Matrices

2.1. Usando `np.matrix`

```
1 import numpy as np
2
3 # Matriz 2x3 creada con matrix
4 A = np.matrix([[1, 2, 3],
5                [4, 5, 6]])
6
7 # Matriz 3x3
8 B = np.matrix([[7, 8, 9],
9                [10, 11, 12],
10               [13, 14, 15]])
11
12 print("A =\n", A)
13 print("B =\n", B)
14 print("Dimensiones de A:", A.shape)
15 print("Tipo:", type(A))
```

Listing 2: Creación de matrices con `np.matrix`

2.2. Usando `np.array`

```
1 import numpy as np
2
3 A = np.array([[1, 2, 3],
4               [4, 5, 6]])
5
6 B = np.array([[7, 8, 9],
7               [10, 11, 12],
8               [13, 14, 15]])
9
10 print("A =\n", A)
11 print("Dimensiones:", A.shape)
```

Listing 3: Creación de matrices con `np.array`

2.3. Matrices especiales

```
1 import numpy as np
2
3 I = np.eye(3)                # Identidad 3x3
4 Z = np.zeros((2, 4))         # Matriz 2x4 de ceros
5 O = np.ones((3, 3))          # Matriz 3x3 de unos
6 D = np.diag([1, 2, 3])       # Matriz diagonal
7
8 print("Identidad:\n", I)
9 print("Ceros:\n", Z)
10 print("Unos:\n", O)
11 print("Diagonal:\n", D)
```

Listing 4: Matrices identidad, de ceros y de unos

3. Operaciones Básicas entre Matrices

3.1. Suma y resta

Sean las matrices

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

```
1 import numpy as np
2
3 A = np.matrix([[1, 2], [3, 4]])
4 B = np.matrix([[5, 6], [7, 8]])
5
6 suma = A + B
7 resta = A - B
8
9 print("A + B =\n", suma)
10 print("A - B =\n", resta)
```

Listing 5: Suma y resta de matrices

El resultado esperado es:

$$A + B = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}, \quad A - B = \begin{pmatrix} -4 & -4 \\ -4 & -4 \end{pmatrix}$$

3.2. Multiplicación elemento a elemento

```
1 import numpy as np
2
3 A = np.matrix([[1, 2], [3, 4]])
4 B = np.matrix([[5, 6], [7, 8]])
5
6 producto = np.multiply(A, B)
7 print("Producto elemento a elemento =\n", producto)
```

Listing 6: Producto elemento a elemento con np.multiply

Resultado:

$$A \odot B = \begin{pmatrix} 5 & 12 \\ 21 & 32 \end{pmatrix}$$

3.3. Multiplicación matricial

Con `np.matrix`, el operador `*` realiza directamente el producto matricial:

```
1 import numpy as np
2
3 A = np.matrix([[1, 2], [3, 4]])
4 B = np.matrix([[5, 6], [7, 8]])
5
6 C = A * B          # Producto matricial con np.matrix
7 print("A * B =\n", C)
```

Listing 7: Multiplicación matricial con `np.matrix`

Con `np.array` se debe usar el operador `@` o `np.dot`:

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 B = np.array([[5, 6], [7, 8]])
5
6 C1 = A @ B          # Operador @
7 C2 = np.dot(A, B)   # Funci n dot
8 C3 = np.matmul(A, B) # Funci n matmul
9
10 print("A @ B =\n", C1)
```

Listing 8: Multiplicación matricial con `np.array`

Resultado:

$$A \cdot B = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

4. Operaciones Avanzadas

4.1. Transpuesta

```
1 import numpy as np
2
3 A = np.matrix([[1, 2, 3], [4, 5, 6]])
4
5 print("A =\n", A)
6 print("A transpuesta =\n", A.T)
```

Listing 9: Transposición de matrices

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

4.2. Determinante

```
1 import numpy as np
2
3 A = np.matrix([[4, 7], [2, 6]])
4
5 det = np.linalg.det(A)
6 print("Determinante:", det)
```

Listing 10: Cálculo del determinante

$$\det(A) = \begin{vmatrix} 4 & 7 \\ 2 & 6 \end{vmatrix} = 4 \cdot 6 - 7 \cdot 2 = 10$$

4.3. Inversa

```

1 import numpy as np
2
3 A = np.matrix([[4, 7], [2, 6]])
4
5 inv = np.linalg.inv(A)
6 print("Inversa de A =\n", inv)
7
8 # Verificacion: A * A^-1 = I
9 print("A * A^-1 =\n", A * inv)

```

Listing 11: Cálculo de la matriz inversa

La verificación debe dar aproximadamente la matriz identidad.

4.4. Rango de una matriz

```

1 import numpy as np
2
3 A = np.matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
4
5 rango = np.linalg.matrix_rank(A)
6 print("Rango:", rango)

```

Listing 12: Rango de una matriz

4.5. Traza

```

1 import numpy as np
2
3 A = np.matrix([[1, 2, 3],
4               [4, 5, 6],
5               [7, 8, 9]])
6
7 traza = np.trace(A)
8 print("Traza:", traza)    # 1 + 5 + 9 = 15

```

Listing 13: Traza de una matriz

5. Resolución de Sistemas de Ecuaciones Lineales

Consideremos el sistema:

$$\begin{cases} 2x + y = 5 \\ x + 3y = 10 \end{cases}$$

En forma matricial: $A\mathbf{x} = \mathbf{b}$, con

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 5 \\ 10 \end{pmatrix}$$

```

1 import numpy as np
2
3 A = np.matrix([[2, 1], [1, 3]])
4 b = np.matrix([[5], [10]])
5
6 # Solucion usando np.linalg.solve
7 sol = np.linalg.solve(A, b)
8 print("Solucion [x, y] =\n", sol)

```

Listing 14: Resolución de un sistema lineal

El resultado es $x = 1$, $y = 3$.

6. Ejemplo Integrador

```

1 import numpy as np
2
3 # Definicion de matrices
4 A = np.matrix([[2, 4, 6],
5                [1, 3, 5],
6                [7, 8, 9]])
7
8 B = np.matrix([[1, 0, 2],
9                [0, 1, 0],
10               [3, 0, 1]])
11
12 # 1. Suma
13 print("A + B =\n", A + B)
14
15 # 2. Producto matricial
16 P = A * B
17 print("A * B =\n", P)
18
19 # 3. Transpuesta del producto
20 print("(A * B)^T =\n", P.T)
21
22 # 4. Determinante de A
23 print("det(A) =", np.linalg.det(A))
24
25 # 5. Rango de B
26 print("Rango(B) =", np.linalg.matrix_rank(B))
27
28 # 6. Inversa de B (si existe)
29 try:
30     invB = np.linalg.inv(B)
31     print("B^-1 =\n", invB)
32 except np.linalg.LinAlgError:
33     print("B no es invertible")

```

Listing 15: Ejemplo completo: operaciones combinadas

7. Resumen de Funciones de NumPy

Función	Descripción
<code>np.matrix()</code>	Crea una matriz 2D
<code>np.array()</code>	Crea un arreglo n-dimensional
<code>np.eye(n)</code>	Matriz identidad $n \times n$
<code>np.zeros(shape)</code>	Matriz de ceros
<code>np.ones(shape)</code>	Matriz de unos
<code>np.diag(v)</code>	Matriz diagonal
<code>A.T</code>	Transpuesta
<code>np.dot(A,B)</code>	Producto matricial
<code>A @ B</code>	Producto matricial (PEP 465)
<code>np.multiply(A,B)</code>	Producto elemento a elemento
<code>np.linalg.det(A)</code>	Determinante
<code>np.linalg.inv(A)</code>	Inversa
<code>np.linalg.solve</code>	Resolver sistema lineal
<code>np.trace(A)</code>	Traza
<code>np.linalg.matrix_rank</code>	Rango

8. Conclusión

La librería NumPy constituye una herramienta fundamental para el cálculo matricial en Python. El uso de `np.matrix` permite una sintaxis compacta para el álgebra lineal, aunque se recomienda migrar a `np.array` con el operador `@` por razones de compatibilidad futura. Las funciones de `numpy.linalg` cubren prácticamente todas las operaciones requeridas en álgebra lineal: determinantes, inversas, resolución de sistemas, rango, valores propios, entre otras.

Referencias

- [1] C. R. Harris et al., *Array programming with NumPy*, Nature, vol. 585, pp. 357–362, 2020.
- [2] NumPy Documentation, <https://numpy.org/doc/stable/>, consultado en 15 de septiembre de 2026.

Operaciones con matrices usando NumPy

1. Introducción

NumPy es una librería fundamental para el cálculo científico en Python. Proporciona un poderoso objeto `ndarray` que permite realizar operaciones vectorizadas de forma eficiente. En este documento se presentan las operaciones matriciales más comunes utilizando NumPy.

2. Instalación e importación

Para usar NumPy, primero debe estar instalado:

```
1 pip install numpy
```

Listing 1: Instalación con pip

Luego se importa comúnmente como `np`:

```
1 import numpy as np
```

Listing 2: Importación de NumPy

3. Creación de matrices

NumPy ofrece varias formas de crear matrices (arreglos bidimensionales).

3.1. Desde una lista de listas

```
1 A = np.array([[1, 2, 3],  
2             [4, 5, 6],  
3             [7, 8, 9]])  
4 print(A)
```

Listing 3: Creación manual

Salida:

```
1 [[1 2 3]  
2  [4 5 6]  
3  [7 8 9]]
```

Listing 4: Salida

3.2. Matrices especiales

```
1 Z = np.zeros((3,3))          # Matriz 3x3 de ceros
2 O = np.ones((2,4))           # Matriz 2x4 de unos
3 I = np.eye(3)                 # Matriz identidad 3x3
4 print("Ceros:\n", Z)
5 print("Unos:\n", O)
6 print("Identidad:\n", I)
```

Listing 5: Matriz de ceros

4. Operaciones básicas

Las operaciones aritméticas (suma, resta, multiplicación elemento a elemento) se realizan con los operadores estándar.

4.1. Suma y resta

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 suma = A + B
5 resta = A - B
6 print("Suma:\n", suma)
7 print("Resta:\n", resta)
```

Listing 6: Suma y resta de matrices

Salida:

```
1 Suma:
2 [[ 6  8]
3  [10 12]]
4 Resta:
5 [[-4 -4]
6  [-4 -4]]
```

Listing 7: Salida

4.2. Multiplicación elemento a elemento (producto de Hadamard)

Se usa el operador `*`.

```
1 prod_hadamard = A * B
2 print("Producto elemento a elemento:\n", prod_hadamard)
```

Listing 8: Producto elemento a elemento

Salida:

```
1 [[ 5 12]
2  [21 32]]
```

4.3. Producto matricial (producto punto)

Para la multiplicación matricial (álgebra lineal) se usa `np.dot()` o el operador `@`.

```
1 producto_matricial = np.dot(A, B)    # o A @ B
2 print("Producto matricial A B:\n", producto_matricial)
```

Listing 9: Producto matricial

Salida:

```
1 [[19 22]
2  [43 50]]
```

4.4. Transpuesta

```
1 A_T = A.T
2 print("Transpuesta de A:\n", A_T)
```

Listing 10: Transpuesta

4.5. Inversa y determinante

Para matrices cuadradas no singulares.

```
1 C = np.array([[4, 7], [2, 6]])
2 det = np.linalg.det(C)
3 invC = np.linalg.inv(C)
4 print("Determinante de C:", det)
5 print("Inversa de C:\n", invC)
6 # Verificación: C * invC = identidad
7 print("C * invC:\n", C @ invC)
```

Listing 11: Inversa y determinante

Salida (aproximada):

```
1 Determinante de C: 10.0
2 Inversa de C:
3  [[ 0.6 -0.7]
4   [-0.2  0.4]]
5 C * invC:
6  [[1.  0.]
7   [0.  1.]]
```

4.6. Resolución de sistemas lineales

NumPy permite resolver sistemas $Ax = b$ mediante `np.linalg.solve`.

```
1 A = np.array([[3, 1], [1, 2]])
2 b = np.array([9, 8])
3 x = np.linalg.solve(A, b)
4 print("Solución del sistema:", x)
5 # Verificación
6 print("A x =", A @ x)
```

Listing 12: Sistema lineal

Salida:

```
1 Solución del sistema: [2. 3.]
2 A x = [9. 8.]
```

4.7. Otras operaciones útiles

- Traza: `np.trace(A)`
- Rango: `np.linalg.matrix_rank(A)`
- Autovalores y autovectores: `np.linalg.eig(A)`
- Normas: `np.linalg.norm(A)`

Ejemplo:

```
1 D = np.array([[1, 2], [3, 4]])
2 print("Traza:", np.trace(D))
3 print("Rango:", np.linalg.matrix_rank(D))
4 eigenvals, eigenvecs = np.linalg.eig(D)
5 print("Autovalores:", eigenvals)
6 print("Autovectores:\n", eigenvecs)
```

Listing 13: Traza y autovalores

5. Conclusión

NumPy proporciona una interfaz sencilla y eficiente para operar con matrices. Las operaciones vectorizadas evitan bucles explícitos y mejoran el rendimiento. Con las funciones del submódulo `linalg` se cubren prácticamente todas las necesidades del álgebra lineal numérica.

Referencias

- Documentación oficial de NumPy: <https://numpy.org/doc/stable/>
- Guía de álgebra lineal con NumPy: <https://numpy.org/doc/stable/reference/routines.linalg.html>


```
import numpy as np
```